

Szoftver Tesztelés Dokumentáció

Egységtesztek kidolgozása Employee Manager rendszerhez

Korpos Botond - Számítástechnika IV A

2026. március 16.

Tartalomjegyzék

1. Bevezetés	3
2. Motivációk	3
3. Telepítés és konfiguráció	3
3.1. Rendszerkövetelmények	3
3.2. Virtuális környezet beállítása	3
3.3. Szükséges könyvtárak telepítése	3
3.4. A tesztek futtatása	4
4. Miért a Pytest?	4
5. A repository felépítése és adatstruktúra	4
5.1. Fájlstruktúra	4
5.2. Az EmployeeManager egységtesztjeinek részletezése	5
5.2.1. test_salary_non_leader függvény	5
5.2.2. test_salary_leader_with_team függvény	6
5.2.3. test_email_sending függvény	7
5.3. A RelationsManager egységtesztjeinek részletezése	8
5.3.1. test_john_doe_data függvény	8
5.3.2. test_john_doe_team_members függvény	9
5.3.3. test_gretchen_salary függvény	10
5.3.4. test_tomas_not_leader függvény	10
5.3.5. test_jude_not_exists függvény	11
6. Összegzés	11

1. Bevezetés

Ez a dokumentáció egy Employee Manager System működését, ennek folyamatát illetve szoftveresen végzett tesztelését mutatja be. A szoftver célja egy vállalati környezet erőforrásainak kezelése, illetve a kapcsolatok, bérkalkulációk automatizálása.

2. Motivációk

A szoftverfejlesztési ciklusban az egységtesztek (Unit Tests) implementálása nem csupán egy opcionális kiegészítés, hanem a stabil és fenntartható kód-alap záloga. Mivel a szoftver alapvető funkciója a fizetések kalkulációja a tesztek automatizált futtatása garantálja, hogy a matematikai képletek minden körülmény között az elvárt eredményt adják. Emelett biztosítja a mozgó dátumok problémájának a kezelését és a regressziós hibák megelőzését is.

3. Telepítés és konfiguráció

3.1. Rendszerkövetelmények

A szoftver futtatásához legalább **Python 3.8** vagy újabb verzió szükséges.

```
1 python --version
```

3.2. Virtuális környezet beállítása

A projekt függőségeinek elkülönítése érdekében javasolt egy virtuális környezet (*Virtual Environment*) használata. Ez biztosítja, hogy a teszteléshez használt könyvtárak ne ütközzenek a rendszer más Python csomagjaival. A virtuális környezet létrehozása és aktiválása Windows környezetben:

```
1 python -m venv venv
2 venv\Scripts\activate
```

3.3. Szükséges könyvtárak telepítése

A projekt a standard Python könyvtárakon kívül a **Pytest** keretrendszert használja a tesztek futtatásához. A telepítés a Python csomagkezelőjével (pip) történik:

```
1 pip install pytest
```

3.4. A tesztek futtatása

Miután a környezet beállítása megtörtént, a tesztek a gyökérkönyvtárból indíthatóak. A `-v` (verbose) kapcsoló használata javasolt a részletesebb kimenet érdekében:

```
1 pytest -v
```

4. Miért a Pytest?

A Pytest a Python ökoszisztéma legnépszerűbb tesztelő eszköze, mivel:

- Egyszerű szintaxist használ (standard `assert` kulcsszó).
- Támogatja a *fixture*-öket az ismétlődő kódok elkerülésére.
- Kiválóan kezelhető vele a *Mocking* (helyettesítő objektumok használata).

5. A repository felépítése és adatstruktúra

A projekt három fő modulból és a hozzájuk tartozó tesztfájlokból áll.

5.1. Fájlstruktúra

A repository az alábbi állományokat tartalmazza:

- `employee.py`: Az alkalmazottak alapvető adatstruktúráját definiáló modul.
- `relations_manager.py`: Az alkalmazottak közötti hierarchiát és az adatlistát kezelő osztály.
- `employee_manager.py`: A bérszámfejtési logikát és az értesítések küldését végző modul.
- `test_employee_manager.py`: Az `EmployeeManager` funkcióit ellenőrző unit tesztek.
- `test_relations.py`: A hierarchiát és adatokat ellenőrző tesztek.

5.2. Az EmployeeManager egységtesztjeinek részletezése

Ebben a szakaszban bemutatásra kerülnek a fizetési logika ellenőrzésére szolgáló teszteseteket, részletezve a bemeneti adatokat és az elvárt eredményeket.

5.2.1. test_salary_non_leader függvény

Ez a teszt egy olyan alkalmazott fizetését ellenőrzi, aki nem rendelkezik vezetői jogosultságokkal, így a fizetése csak az alapbérből és a munkaviszony hossza után járó bónuszból tevődik össze.

```
1 def test_salary_non_leader(manager, mock_rm):
2     base_salary = 1000
3     hire_date = datetime.date(1998, 10, 10)
4
5     emp = Employee(id=10, first_name="Test", last_name="
6         User",
7                     birth_date=datetime.date(1970, 1, 1),
8                     base_salary=base_salary,
9                     hire_date=hire_date)
10
11     mock_rm.is_leader.return_value = False
12
13     current_year = datetime.date.today().year
14     expected_years = current_year - hire_date.year
15     expected_salary = base_salary + (expected_years *
16         100)
17
18     result = manager.calculate_salary(emp)
19
20     assert result == expected_salary
```

Listing 1. Nem vezető alkalmazott fizetésének tesztelése

Teszt leírása és paraméterezése:

- **Bemeneti adatok (Input):**
 - Alapbér: 1000\$
 - Belépés dátuma: 1998.10.10.
 - Pozíció: Beosztott (a mock_rm.is_leader hamis értéket ad vissza).
- **Alkalmazott logika:** A szoftver kiszámítja az aktuális év és a belépési év különbségét, majd minden ledolgozott év után 100\$ bónuszt ad az alapbérhez.

- **Dinamikus elvárt kimenet (Expected Output):** Mivel a teszt 2026-ban fut, a munkaviszony hossza $2026 - 1998 = 28$ év. Így az elvárt összeg: $1000 + (28 \times 100) = 3800\$$.

5.2.2. test_salary_leader_with_team függvény

Ez a teszt eset egy olyan csapatvezető fizetését ellenőrzi, akinek az alapbérén felül a munkaviszony hossza és a menedzselte csapat létszáma után is bónusz jár.

```

1  def test_salary_leader_with_team(manager, mock_rm):
2      base_salary = 2000
3      hire_date = datetime.date(2008, 10, 10)
4      team_count = 3
5
6      emp = Employee(id=11, first_name="Leader", last_name=
7                  "User",
8                      birth_date=datetime.date(1970, 1, 1),
9                      base_salary=base_salary,
10                     hire_date=hire_date)
11
12     mock_rm.is_leader.return_value = True
13     mock_rm.get_team_members.return_value = [1, 2, 3]
14
15     current_year = datetime.date.today().year
16     expected_years = current_year - hire_date.year
17     expected_salary = base_salary + (expected_years *
18                               100) + (team_count * 200)
19
20     result = manager.calculate_salary(emp)
21
22     assert result == expected_salary

```

Listing 2. Csapatvezetői fizetés tesztelése bónuszokkal

Teszt leírása és paraméterezése:

- **Bemeneti adatok (Input):**
 - Alapbér: 2000\$
 - Belépés dátuma: 2008.10.10.
 - Pozíció: Vezető (`is_leader = True`).
 - Csapatlétszám: 3 fő (szimulálva a `[1, 2, 3]` listával).

- **Alkalmazott logika:** A fizetés három komponensből áll össze:
 1. Fix alaphér.
 2. Régiségi bónusz: minden ledolgozott év után 100\$.
 3. Vezetői bónusz: minden csapattag után 200\$.
- **Dinamikus elvárt kimenet (Expected Output):** 2026-os futtatás esetén a munkaviszony $2026 - 2008 = 18$ év.
Számítás: $2000 + (18 \times 100) + (3 \times 200) = 2000 + 1800 + 600 = 4400\$$.

5.2.3. test_email_sending függvény

Ez a teszt azt hivatott ellenőrizni, hogy a bérkalkuláció után a rendszer megfelelően összeállítja-e és kiküldi-e az értesítést az alkalmazott számára.

```

1 def test_email_sending(manager, mock_rm):
2     emp = Employee(id=1, first_name="John", last_name="
      Doe",
3                               birth_date=datetime.date(1970, 1, 1),
4                               base_salary=3000,
5                               hire_date=datetime.date(2010, 1, 1))
6
7     mock_rm.is_leader.return_value = False
8
9     with patch('builtins.print') as mock_print:
10         manager.calculate_salary_and_send_email(emp)
11
12         mock_print.assert_called()
13
14         args, kwargs = mock_print.call_args
15         actual_message = args[0]
16
17         assert "John Doe" in actual_message
18         assert "your salary" in actual_message
19
20         salary = manager.calculate_salary(emp)
21         assert str(salary) in actual_message

```

Listing 3. Email értesítés küldésének és tartalmának ellenőrzése

Teszt leírása és paraméterezése:

- **Bemeneti adatok (Input):** Egy standard alkalmazott ("John Doe"), 3000\$ alaphérrel és 2010-es belépéssel.

- **Vizsgált viselkedés:** A `calculate_salary_and_send_email` metódus meghívása.
- **Elvárt kimenet (Expected Output):**
 - A beépített `print` függvénynek legalább egyszer le kell futnia.
 - Az üzenetnek tartalmaznia kell az alkalmazott nevét ("John Doe").
 - Az üzenetnek tartalmaznia kell egy sablon szöveget ("your salary").
 - Az üzenetnek tartalmaznia kell a dinamikusan kiszámolt aktuális fizetést karakterlánc formátumban.

5.3. A RelationsManager egységtesztjeinek részletezése

Míg az `EmployeeManager` tesztjei az üzleti logikára fókuszálnak, a `RelationsManager` tesztjei az adatok helyességét és a hierarchikus kapcsolatok integritását ellenőrzik a rendszerben.

5.3.1. test_john_doe_data függvény

Ez a teszt azt ellenőrzi, hogy a rendszer adatbázisában szerepel-e a specifikációban megadott kulcsszereplő, és az ő adatai megfelelnek-e az elvárásoknak.

```

1 def test_john_doe_data(rm):
2     """
3     Check if there is a team leader called John Doe whose
4     birthdate is 31.01.1970.
5     """
6     john = next((e for e in rm.get_all_employees() if e.
7                  first_name == "John" and e.last_name == "Doe"),
8                 None)
9     assert john is not None
10    assert john.birth_date == datetime.date(1970, 1, 31)
11    assert rm.is_leader(john) is True

```

Listing 4. Alkalmazotti adatintegritás ellenőrzése

Teszt leírása és paraméterezése:

- **Bemeneti adatok (Input):** A `RelationsManager` által kezelt teljes alkalmazotti lista.

- **Vizsgált folyamat:**

1. Keresés az alkalmazottak között kereszt- és vezetéknév alapján (John Doe).
2. A keresési találat meglétének ellenőrzése.
3. A születési dátum és a vezetői státusz validálása.

- **Elvárt kimenet (Expected Output):**

- Az objektum nem lehet None.
- A születési dátumnak pontosan 1970.01.31-nek kell lennie.
- Az `is_leader` metódusnak True értéket kell visszaadnia az adott objektumra.

5.3.2. `test_john_doe_team_members` függvény

Ez a teszt a szervezeti hierarchia pontosságát vizsgálja, ellenőrizve, hogy a kijelölt vezetőhöz a megfelelő beosztottak vannak-e rendelve, illetve kizárja a nem odatartozó személyeket.

```
1 def test_john_doe_team_members(rm):
2     john = next(e for e in rm.get_all_employees() if e.id
3                 == 1)
4     team_ids = rm.get_team_members(john)
5     team_names = [f"{e.first_name} {e.last_name}" for e
6                   in rm.get_all_employees() if e.id in team_ids]
7     assert "Myrta Torkelson" in team_names
8     assert "Jettie Lynch" in team_names
9     assert "Tomas Andre" not in team_names
```

Listing 5. Csapatvezetői kapcsolatok ellenőrzése

Teszt leírása és paraméterezése:

- **Bemeneti adatok (Input):** John Doe alkalmazott (ID: 1) és a teljes kapcsolati háló.
- **Vizsgált folyamat:** A vezetőhöz tartozó csapat-ID listájának lekérése, majd a nevek feloldása.
- **Elvárt kimenet (Expected Output):**

- Myrta Torkelson és Jettie Lynch szerepelnek a listában.
- Tomas Andre neve nem szerepel John Doe csapattagjai között.

5.3.3. test_gretchen_salary függvény

Egyszerűbb adatintegritási teszt, amely egy konkrét alkalmazott specifikus béradatát ellenőrzi a rendszerben.

```

1 def test_gretchen_salary(rm):
2     gretchen = next(e for e in rm.get_all_employees() if
3         e.first_name == "Gretchen")
4     assert gretchen.base_salary == 4000

```

Listing 6. Konkrét alkalmazott alaphérének ellenőrzése

Teszt leírása és paraméterezése:

- **Bemeneti adatok (Input):** Gretchen Walford rekordja az alkalmazotti listából.
- **Elvárt kimenet (Expected Output):** Az alaphér pontosan 4000\$.

5.3.4. test_tomas_not_leader függvény

Ez a teszt a negatív logikai ágakat vizsgálja: mit történik, ha olyan személy csapattagjait próbáljuk lekérdezni, aki nem vezető.

```

1 def test_tomas_not_leader(rm):
2     tomas = next(e for e in rm.get_all_employees() if e.
3         first_name == "Tomas")
4     assert rm.is_leader(tomas) is False
5     assert rm.get_team_members(tomas) is None

```

Listing 7. Nem vezető státusz és csapatlekérdezés tesztelése

Teszt leírása és paraméterezése:

- **Bemeneti adatok (Input):** Tomas Andre alkalmazott adatai.
- **Elvárt kimenet (Expected Output):**
 - Az `is_leader` metódus hamis (`False`) értéket ad.
 - A `get_team_members` metódus hívása eredménytelen (`None`).

5.3.5. test_jude_not_exists függvény

A teszt azt ellenőrzi, hogy a rendszer ne tartalmazzon olyan adatokat, amelyeknek nem szabadna a nyilvántartásban lenniük.

```
1 def test_jude_not_exists(rm):  
2     all_names = [f"{e.first_name} {e.last_name}" for e in  
                 rm.get_all_employees()]  
3     assert "Jude Overcash" not in all_names
```

Listing 8. Nem létező rekord keresésének tesztelése

Teszt leírása és paraméterezése:

- **Bemeneti adatok (Input):** A rendszer összes regisztrált alkalmazottjának listája.
- **Elvart kimenet (Expected Output):** "Jude Overcash" neve nem található meg az összesített névlistában.

6. Összegzés

A projekt során egy komplex alkalmazotti és fizetés számoló rendszer egységtesztelését valósítottuk meg a **Pytest** keretrendszer segítségével. A fejlesztési folyamat során kiemelt figyelmet fordítottunk a modularitásra és a tesztek fenntarthatóságára.

A dokumentációban bemutatott tesztesetek lefedik a rendszer legkritikusabb funkcióit:

- Az adatintegritást ellenőrző tesztek garantálják, hogy a forrásadatok (nevek, születési dátumok, alaphérek) pontosan kerülnek betöltésre.
- A hierarchia-vizsgálatok biztosítják a szervezeti struktúra és a vezető-beosztott kapcsolatok helyességét.
- A bérkalkulációs algoritmusok tesztelése során alkalmazott **dinamikus elvart érték számítás** és a **Mocking** technológia lehetővé teszi, hogy a rendszer az idő múlásától és a külső függőségektől függetlenül is precíz eredményt adjon.

A tesztelési folyamat eredményeként egy olyan stabil kódalap jött létre, amely minimálisra csökkenti a manuális ellenőrzés szükségességét, és hatékony védelmet nyújt a jövőbeli fejlesztések során fellépő regressziós hibák

ellen. Az automatizált unit tesztek megléte nemcsak a szoftver minőségét javítja, hanem alapvető bizalmat ad a bérszámfejtési folyamatok pontosságát illetően is.